



Kafka

Kafka

Kafka é uma plataforma de Código aberto de processamento de streams desenvolvida pela Apache Software Foundation, escrita em Scala e Java. O projeto tem como objetivo fornecer uma plataforma unificada, de alta capacidade e baixa latência para tratamento de dados em tempo real. Sua camada de armazenamento é, essencialmente, uma "fila de mensagens de *publishers/subscribers* altamente escalável projetada como um log de transações distribuído.

- ▶ Características-chave:
 - ▶ Altamente escalável;
 - ▶ Tolerante a falhas;
 - ▶ Permite o desacoplamento entre produtor e consumidor;

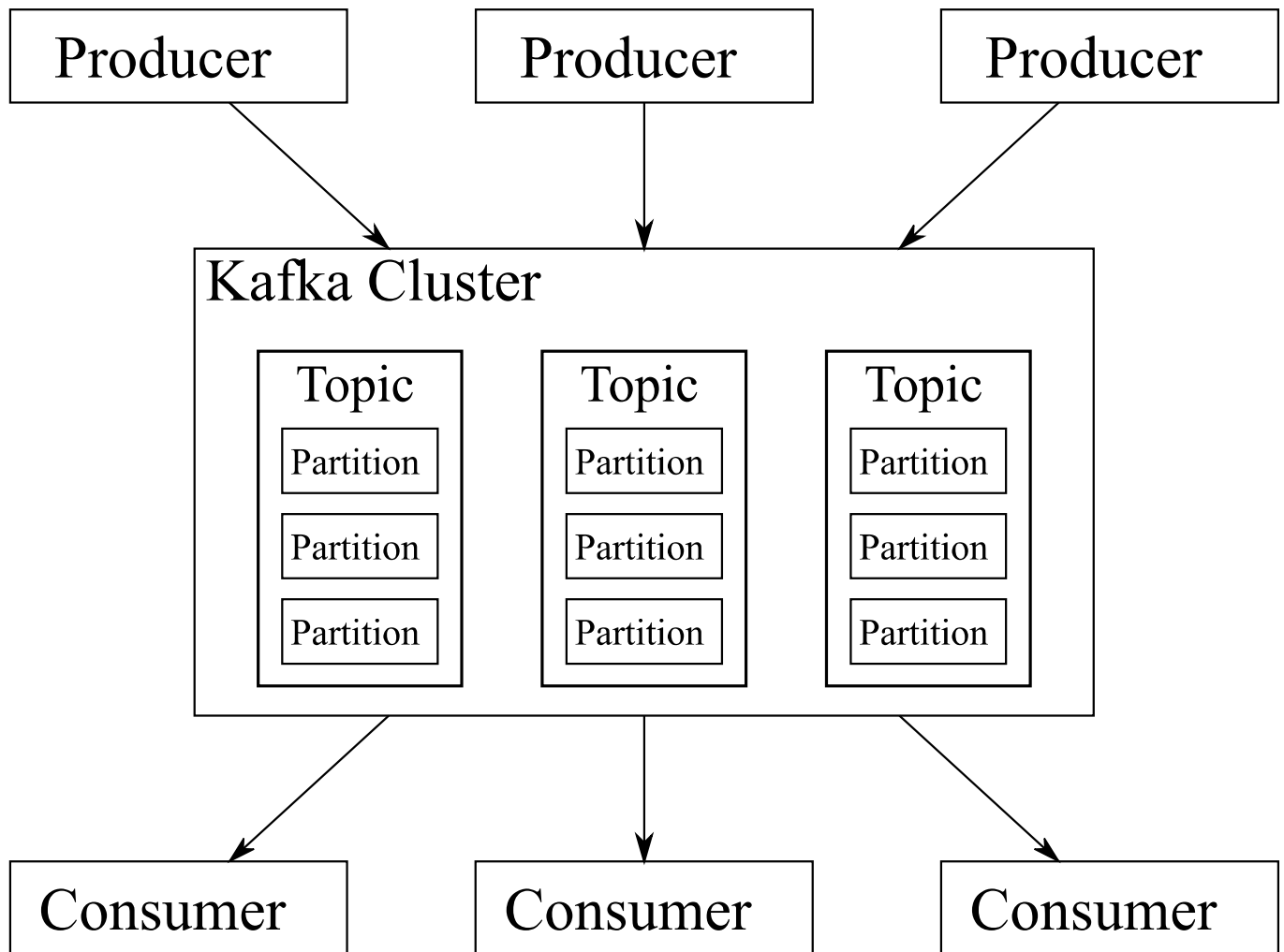


História

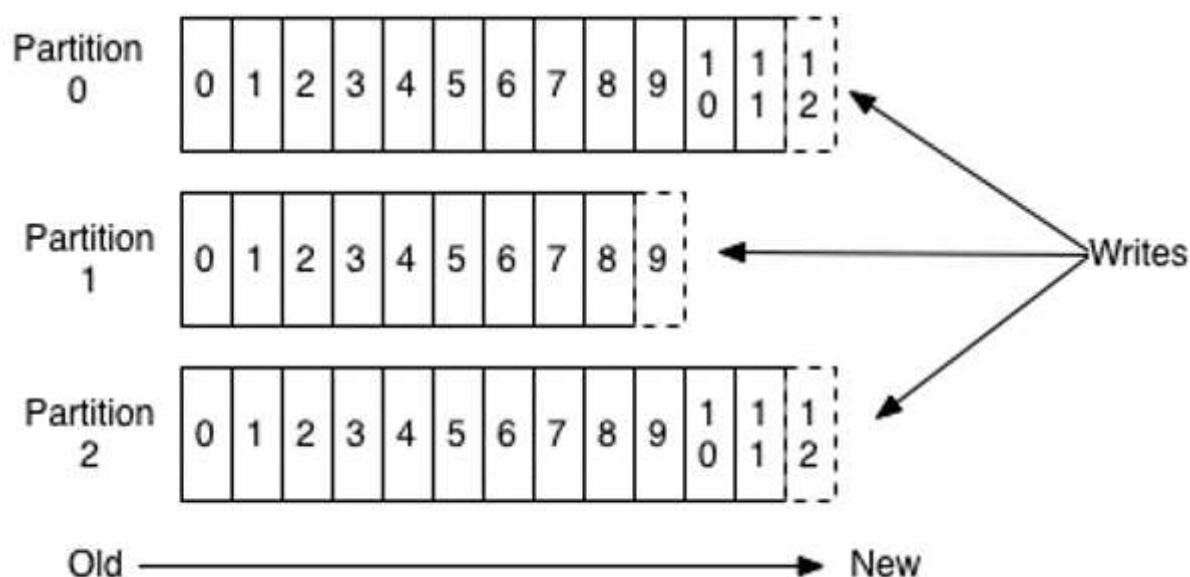
O Apache Kafka foi originalmente desenvolvido pelo LinkedIn, e teve posteriormente, seu código aberto no início de 2011, os desenvolvedores criaram uma nova empresa chamada Confluent com foco em Kafka. Seu nome faz referência ao autor Franz Kafka, porque ele é "um sistema otimizado para escrever", e os criadores gostavam do trabalho do escritor tcheco.

Arquitetura e conceitos

O Apache Kafka é baseado no *commit log*, ele permite que os usuários inscrevam-se e publiquem dados para qualquer número de sistemas ou aplicações em tempo real. O Kafka armazena mensagens por chave-valor que provêm de inúmeros processos chamados de "produtores". Os dados podem, assim, ser particionados em diferentes "partições" e diferentes "tópicos". Dentro de uma partição, as mensagens são estritamente ordenadas pela sua posição (a posição de uma mensagem dentro de uma partição), e indexados e



- **Broker:** Nó (servidor) do cluster Kafka;
- **Produtor:** Publica (ou “empurra”) mensagens em um ou mais tópicos Kafka;
- **Consumidor:** Interessado que tem mensagens “puxadas” de um tópico Kafka;
- **Tópico:** Agrupamento de mensagens imutáveis e ordenadas dentro do Kafka. Pode ter muitos “escritores” e “leitores”
 - Particionamento: Um tópico pode ser particionado em um ou mais brokers;
 - Replicação: Um tópico pode ser replicado em um ou mais brokers;



- **Retenção:** Persistência dos dados em um determinado tópico, pode ser baseado em tempo ou quantidade de dados
- **Zookeeper:** Componente que armazena os metadados do Kafka, incluindo o Offset

Padrão de Nomenclatura de tópicos

ONS.<SiglaAplicacao>.<NomeEvento ou NomeProcesso>.
<EstagioProcesso(Opcional)>.<Ambiente (Apenas para teste - Local/Snd/Tst/Dsv)>.<Versao(Opcional. Ex.:v1, v2, v3...)>

- Apenas para ambiente de teste,

Exemplos de código

C#

Utilizar pacote Nuget "Cloudera.Kafka"

Produtor:

```

1 | var _producerConfig = new ProducerConfig
2 | {
3 |     //Servidores do cluster, separados por vírgula, usar a chave
4 |     "Kafka_BootstrapServers" do ConfigIT
5 |     BootstrapServers = "tst-kafka-001.ons.org.br:9093,tst-kafka-
6 |

```



```

9      //Usar a chave "Kafka_SecurityProtocol" do ConfigIT
10     SecurityProtocol = (SecurityProtocol)Enum.Parse(typeof(SecurityProtocol),
11     "SaslSsl"),
12
13     //Usar chave "Kafka_SslCaPem" do ConfigIT
14     //Para Configuração em XML (web.config), substituir \n por <enter>, como
15     exemplo abaixo
16     //SslCaPem = ConfigurationManager.AppSettings["<Chave
17     certificado>"].Replace("\\n", System.Environment.NewLine);
18     SslCaPem = "<Conteúdo certificado>",
19
20     //Usar chave "Kafka_SslKeystorePassword" do ConfigIT
21     SslKeystorePassword = "<Senha certificado>",
22
23     //Usar chave "Kafka_SaslMechanism" do ConfigIT
24     SaslMechanism = (SaslMechanism)Enum.Parse(typeof(SaslMechanism), "Plain"),
25
26     //Usar usuário de serviço da sua aplicação com permissões
27     //Atenção!!! Usuário é case sensitive!!!
28     SaslUsername = "UsuarioAplicacao",
29     SaslPassword = "SenhaAplicacao"
30 };
31 var _builderProducer = new ProducerBuilder<string, string>
32 (_producerConfig).Build();
33 var mensagem = new Message<string, string>() { Key = "Chave Mensagem", Value
34 = "Payload mensagem" };
35 //Inserir cabeçalhos importantes para filtrar mensagens no consumidor
36 mensagem.Headers = new Headers();
37 mensagem.Headers.Add(new Header("Chave", Encoding.UTF8.GetBytes("Valor")));
38 mensagem.Headers.Add(new Header("Chave2",
39 Encoding.UTF8.GetBytes("Valor2")));
40 //Respeitar o padrão de nomenclatura de tópicos
41 _builderProducer.Produce("ONS.AppExemplo.MeuProdutoTeste.Consolidacao.v1",
42 mensagem);

```

Consumidor:

```

1  ConsumerConfig _consumerConfig = new ConsumerConfig
2  {
3      //Servidores do cluster, separados por vírgula, usar a chave
4      "Kafka_BootstrapServers" do ConfigIT
5      BootstrapServers = "tst-kafka-001.ons.org.br:9093,tst-kafka-
6      002.ons.org.br:9093,tst-kafka-003.ons.org.br:9093",
7
8      //Usar a chave "Kafka_SecurityProtocol" do ConfigIT
9      SecurityProtocol = (SecurityProtocol)Enum.Parse(typeof(SecurityProtocol),
10     "SaslSsl"),

```



```

13 //Atenção!!! Usuário é case sensitive!!!
14 SaslUsername = "UsuarioAplicacao",
15 SaslPassword = "SenhaAplicacao",
16
17 //Usar chave "Kafka_SslCaPem" do ConfigIT
18 //Para Configuração em XML (web.config), substituir \n por <enter>, como
19 exemplo abaixo
20 //SslCaPem = ConfigurationManager.AppSettings["<Chave
21 certificado>"].Replace("\\n", System.Environment.NewLine);
22 SslCaPem = "<Conteúdo certificado>",
23
24 //Usar chave "Kafka_SslKeystorePassword" do ConfigIT
25 SslKeystorePassword = "<Senha Certificado>",
26
27 //Usar chave "Kafka_SaslMechanism" do ConfigIT
28 SaslMechanism = (SaslMechanism)Enum.Parse(typeof(SaslMechanism), "Plain"),
29
30 //O GroupId informa um grupo de consumidores, repetir o nome do grupo para
31 aplicações distribuídas
32 GroupId = "<NomeGrupo>",
33
34 //Habilita o Commit do cliente para o server automático do offset, caso
35 seja falso, deve ser feito o commit pelo código do usuário
36 EnableAutoCommit = true,
37
38 //Habilita o armazenamento do offset de forma automática dentro do cliente
39 consumidor
40 EnableAutoOffsetStore = true,
41
42 //"Posição" inicial do Offset quando o "ConsumerBuilder" for instanciado
43 //Earliest = Primeira posição disponível no tópico
44 //Latest = última posição no tópico
45 //Error = última posição gravada para o grupo pelo Zookeeper
46 AutoOffsetReset = AutoOffsetReset.Earliest
47 };
48
49
50 using (var c = new ConsumerBuilder<string, string>(_consumerConfig).Build())
51 {
52     //Topico(s) que o consumidor está inscrito, suporta o caracter coringa
53     "*", utilizando o prefixo "^" Ex.: "^ONS*"
54     c.Subscribe("ONS.AppExemplo.MeuProdutoTeste.Consolidacao.v1");
55     var cts = new CancellationTokenSource();
56     try
57     {
58         //Colocar em Thread separada
59         while (true)
60         {
61             //Retornando mensagem quando pausa no(s) tópico(s) inscrito
62

```



```

65 | consumidor
66 |         var cabecalho = message.Message.Headers.FirstOrDefault(x =>
67 | x.Key == "Chave2");
        if (cabecalho != null &&
        Encoding.UTF8.GetString(cabecalho.GetValueBytes()).Equals("Valor"))
        {
            //Escrevendo valor da mensagem
            Console.WriteLine(message.Message.Value);
        }
    }
    catch (OperationCanceledException)
    {
        c.Close();
    }
}

```

Python

Utilizar pacote PIP "kafka-python"

Consumidor:

```

1 | import sys
2 | from os import path
3 | from pprint import pprint
4 | from traceback import print_exc
5 | from kafka import KafkaConsumer
6 |
7 | kafkaBrokers= 'tst-kafka-001.ons.org.br:9093,tst-kafka-
8 | 002.ons.org.br:9093,tst-kafka-003.ons.org.br:9093'
9 | NomeTopico='ONS.AppExemplo.MeuProdutoTeste.Consolidacao.v1'
10 | username='<usuarioAplicacao>'
11 | password='<SenhaAplicacao>'
12 | SenhaCertificado= '<senhaCertificado>'
13 | PathCertificado='C:\\<Path>\\<certificadoAmbiente>.pem'
14 | PathCACertificado='C:\\<Path>\\<certificadoAmbiente>.pem'
15 | ProtocoloSeguranca=' SASL_SSL '
16 | Mecanismo = 'PLAIN'
17 | try:
18 |     consumer = KafkaConsumer(bootstrap_servers=kafkaBrokers,
19 |                             security_protocol=ProtocoloSeguranca,
20 |                             sasl_plain_username=username,
21 |                             sasl_plain_password=password,
22 |                             ssl_certfile = PathCertificado,
23 |                             ssl_cafile = PathCACertificado,

```



```

20
27     consumer.subscribe(NomeTopico)
28     for msg in consumer:
29         print(msg)
30         consumer.close()
31 except Exception as e:
32     print_exc()
    print(e)

```

Servidores

Ambiente	URL de administração	Servidores Cluster
Desenvolvimento/Teste	https://tst-kafka-001:7183/ 	▸ tst-kafka-001.ons.org.br ▸ tst-kafka-002.ons.org.br ▸ tst-kafka-003.ons.org.br
Homologação	https://hom-kafka-11:7183/ 	▸ hom-kafka-11.ons.org.br ▸ hom-kafka-21.ons.org.br ▸ hom-kafka-31.ons.org.br
Produção	https://prd-kafka-11:7183/ 	▸ prd-kafka-11.ons.org.br ▸ prd-kafka-21.ons.org.br ▸ prd-kafka-31.ons.org.br ▸ prd-kafka-41.ons.org.br ▸ prd-kafka-51.ons.org.br ▸ prd-kafka-61.ons.org.br ▸ prd-kafka-61.ons.org.br ▸ prd-kafka-71.ons.org.br

Como obter o certificado para autenticação

Os certificados e senhas estão disponíveis no [Configit](#) , mas caso seja necessário, é possível obter o arquivo `cm-auto-global_cacerts.pem` no diretório `/var/lib/cloudera-scm-agent/agent-cert` nos servidores Kafka. Para obter a senha do certificado, utilize um navegador já autenticado e acesse a seguinte

URL: <https://<servidor>:7183/api/v44/certs/truststorePassword>

Chaves ConfigIT

Chave	Descrição
<code>Kafka_BootstrapServers</code>	Servidores do Cluster, separados por vírgula



Kafka_SslKeystorePassword	Senha do certificado
----------------------------------	-----------------------------

Ligações Externas

- [Apache Kafka](#) 
- [Kafka vs ESB](#) 
- [Kafka @ Medium](#) 
- [Entendendo Kafka](#) 
- [Kafka Developer](#) 
- [Kafka definitive Guide](#) 

Distribuído por [Wiki.js](#)